

Windowing range adaptors: `views::chunk` and `views::slide`

Document #: P2442R1
Date: 2021-12-05
Project: Programming Language C++
Audience: LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

[1 Abstract](#)

[2 Revision history](#)

[3 Example](#)

[4 Design](#)

[4.1 chunk](#)

[4.1.1 Input range support](#)

[4.1.2 Value type](#)

[4.1.3 Conditionally common](#)

[4.1.4 Conditionally borrowed](#)

[4.1.5 Implementation notes](#)

[4.2 slide](#)

[4.2.1 Forward ranges only](#)

[4.2.2 Value type](#)

[4.2.3 Conditionally common and borrowed](#)

[4.2.4 Implementation notes](#)

[4.3 Implementation experience](#)

[5 Wording](#)

[5.1 Addition to `<ranges>`](#)

[5.2 chunk](#)

[24.7.? Chunk view \[range.chunk\]](#)

[24.7.?.1 Overview \[range.chunk.overview\]](#)

[24.7.?.2 chunk view for input ranges \[range.chunk.view.input\]](#)

[24.7.?.3 Class chunk view::outer-iterator \[range.chunk.outer.iter\]](#)

[24.7.?.4 Class chunk view::outer-iterator::value_type \[range.chunk.outer.value\]](#)

[24.7.?.5 Class chunk view::inner-iterator \[range.chunk.inner.iter\]](#)

[24.7.?.6 chunk view for forward ranges \[range.chunk.view.fwd\]](#)

[24.7.?.7 Class template chunk view<V>::iterator for forward ranges \[range.chunk.fwd.iter\]](#)

[5.3 slide](#)

[24.7.? Slide view \[range.slide\]](#)

[24.7.?1 Overview \[range.slide.overview\]](#)

[24.7.?2 Class template slide view \[range.slide.view\]](#)

[24.7.?3 Class template slide view::iterator \[range.slide.iterator\]](#)

[24.7.?4 Class slide view::sentinel \[range.slide.sentinel\]](#)

[5.4 Feature-test macro](#)

[6 References](#)

§ 1 Abstract

This paper proposes two range adaptors, `views::chunk` and `views::slide`, as described in section 3.5 of [\[P2214R0\]](#).

§ 2 Revision history

- R1:
 - Split feature test macro into two as requested by LEWG.
 - Incorporated LWG review feedback on 2021-11-19 and 2021-11-26.

§ 3 Example

```
std::vector v = {1, 2, 3, 4, 5};  
fmt::print("{}\n", v | std::views::chunk(2)); // [[1, 2], [3, 4], [5]]  
fmt::print("{}\n", v | std::views::slide(2));  // [[1, 2], [2, 3], [3, 4], [4, 5]]
```

§ 4 Design

Both of these range adaptors are well-known quantities that have been shipped in range-v3 for years and are further discussed in section 3.5 of [\[P2214R0\]](#). The discussion below assumes familiarity with that paper.

§ 4.1 chunk

The basic idea of `chunk` is simple: `views::chunk(R, N)` divides `R` into non-overlapping `N`-sized chunks, except that the last chunk can be smaller than `N` in size. It is a precondition that `N` is positive.

§ 4.1.1 Input range support

Matching the range-v3 implementation, the proposed `chunk` supports input ranges, even though the algorithm necessary for such support is significantly different.

In particular, for input ranges, the underlying iterator as well as related iteration state is tracked in the `chunk_view` object itself. This means that this `chunk_view` can only support non-`const` iteration. As a point of departure from range-v3, both outer and inner iterators are move-only input iterators.

Because the inner iterator and outer iterator share state, and moving from the stored underlying iterators can invalidate both iterators, only the non-mutating `base()` `const&` overload is provided for the inner iterator to avoid this sort of spooky-action-at-a-distance invalidation:

```
auto v = some_input_view | views::chunk(2);
auto outer = v.begin();
auto range = *outer;
range.begin().base(); // if this moved the underlying iterator, outer would be
                       invalidated
```

Providing `base()` for the outer iterator would be misleading as the stored iterator mutates when the inner range is being iterated over.

§ 4.1.2 Value type

For input ranges, `chunk` has a bespoke value type that is itself an input view. For forward and stronger ranges, `chunk` defers to `views::take` for its value type.

§ 4.1.3 Conditionally common

In range-v3, `chunk` is never a common range. `chunk` as proposed here is a common range if the underlying range is forward, common, and either sized or non-bidirectional.

For bidirectional and stronger ranges, the need to size the last chunk correctly from the end iterator requires the underlying range to be sized.

§ 4.1.4 Conditionally borrowed

As with range-v3, this paper proposes that `chunk` is borrowed if the underlying view is forward and borrowed.

§ 4.1.5 Implementation notes

For input ranges, `chunk_view` stores

- the current underlying iterator (*current_*); and
- the number of elements left in the current chunk (*remainder_*).

Incrementing the inner iterator increments *current_* and decrements *remainder_*, setting it to zero if the end of the chunk is reached.

Incrementing the outer iterator increments *current_ remainder_* times so that we start at the next *N*-element chunk even if the inner view isn't iterated to end, and then sets *remainder_* to the chunk size.

For forward and stronger ranges, `chunk_view`'s *iterator* stores an exposition-only data member *missing_*. This data member can only be nonzero when the iterator is the past-the-end iterator, in which case it represents the difference between *N* and the size of the last chunk.

§ 4.2 slide

`views::slide(R, N)` produces a range of ranges such that the M -th range is a view into the M -th through $(M+N-1)$ -th elements of R . It is similar to `views::adjacent` ([P2321R2]), except that the size of the window N is provided at runtime. It is a precondition that N is positive.

§ 4.2.1 Forward ranges only

Unlike `chunk`, and similar to `adjacent`, `slide` does not support input ranges. It might be possible to support a window size of 1 - but then users can just use `chunk` instead. Caching elements is not considered a viable approach, essentially for the reasons discussed in section 4.3.4 of [P2321R2].

§ 4.2.2 Value type

`slide` defers to `views::counted` for its value type.

§ 4.2.3 Conditionally common and borrowed

`slide` is a common range if the underlying range is (or can be trivially made one), and is a borrowed range if the underlying range is.

§ 4.2.4 Implementation notes

There are basically two strategies for `slide`. For simplicity the discussion below ignores the case where the number of elements in the source view is less than N .

- When the end iterator of the underlying range can be known in constant time and is at least bidirectional, `slide` can go back $N - 1$ elements from that iterator to find the first iterator that can't start a range. In this case, `slide`'s iterator need only store the first iterator of the window `i` and the window size N .
- Otherwise, `slide`'s iterator must also store the iterator to the last element in the window (that is, `i + (N - 1)`). When that iterator compares equal to the end of the source range, the iterator is past-the-end (since we can no longer produce a range of N elements).

§ 4.3 Implementation experience

Both `chunk` and `slide` (under the name `sliding`) have been implemented in `range-v3`, and much of the subtler aspects of the implementation logic in the wording below are taken from there (any errors are mine, of course, though hopefully there isn't any).

I also have implemented a version that directly follows the proposed wording below without issue.

§ 5 Wording

§ 5.1 Addition to `<ranges>`

Add the following to 24.2 [ranges.syn], header `<ranges>` synopsis:

```

// [...]
namespace std::ranges {
    // [...]

    // [range.chunk], chunk view
    template<view V>
        requires input_range<V>
    class chunk_view;

    template<view V>
        requires forward_range<V>
    class chunk_view<V>;

    template<class V>
        inline constexpr bool enable_borrowed_range<chunk_view<V>> =
            forward_range<V> && enable_borrowed_range<V>;

    namespace views {
        inline constexpr unspecified chunk = unspecified;
    }

    // [range.slide], slide view
    template<view V>
        requires forward_range<V>
    class slide_view;

    template<class V>
        inline constexpr bool enable_borrowed_range<slide_view<V>> =
            enable_borrowed_range<V>;

    namespace views {
        inline constexpr unspecified slide = unspecified;
    }
}

```

§ 5.2 chunk

Add the following subclause to 24.7 [\[range.adaptors\]](#).

§ 24.7.? Chunk view [\[range.chunk\]](#)

§ 24.7.?.1 Overview [\[range.chunk.overview\]](#)

- 1 chunk_view takes a view and a number N and produces a range of views that are N -sized non-overlapping successive chunks of the elements of the original view, in order. The last view in the range can have fewer than N elements.
- 2 The name `views::chunk` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions E and N , the expression `views::chunk( $E$ ,  $N$ )` is expression-equivalent to `chunk_view( $E$ ,  $N$ )`.

[Example 1:

```
vector v = {1, 2, 3, 4, 5};

for (auto r : v | views::chunk(2)) {
    cout << '[';
    auto sep = "";
    for(auto i : r) {
        cout << sep << i;
        sep = ", ";
    }
    cout << "]" << " ";
}
// prints: [1, 2] [3, 4] [5]
```

— end example]

§ 24.7.2 chunk_view for input ranges [range.chunk.view.input]

```
namespace std::ranges {

template<class I>
constexpr I div-ceil(I num, I denom) { // exposition only
    I r = num / denom;
    if (num % denom) {
        ++r;
    }
    return r;
}

template<view V>
requires input_range<V>
class chunk_view : public view_interface<chunk_view<V>>{
    V base_ = V(); // exposition only
    range_difference_t<V> n_ = 0; // exposition only
    range_difference_t<V> remainder_ = 0; // exposition only

    non-propagating-cache<iterator_t<V>> current_; // exposition only

    // [range.chunk.outer.iter], class chunk_view::outer-iterator
    class outer-iterator; // exposition only
    // [range.chunk.inner.iter], class chunk_view::inner-iterator
    class inner-iterator; // exposition only

public:
    chunk_view() requires default_initializable<V> = default;
    constexpr explicit chunk_view(V base, range_difference_t<V> n);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr outer-iterator begin();
    constexpr default_sentinel_t end() noexcept;

    constexpr auto size() requires sized_range<V>;
    constexpr auto size() const requires sized_range<const V>;
};
```

```
template<class R>
    chunk_view(R&& r, range_difference_t<R>) -> chunk_view<views::all_t<R>>;
}
```

```
constexpr explicit chunk_view(V base, range_difference_t<V> n);
```

- 1 *Preconditions:* `n > 0` is true.
- 2 *Effects:* Initializes `base_` with `std::move(base)` and `n_` with `n`.

```
constexpr outer-iterator begin();
```

- 3 *Effects:* Equivalent to:


```
current_ = ranges::begin(base_);
remainder_ = n_;
return outer-iterator(*this);
```

```
constexpr default_sentinel_t end() noexcept;
```

- 4 *Returns:* `default_sentinel`.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

- 5 *Effects:* Equivalent to:


```
return to-unsigned-like(div-ceil(ranges::distance(base_), n_));
```

§ 24.7.3.3 Class `chunk_view::outer-iterator` [range.chunk.outer.iter]

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        class chunk_view<V>::outer-iterator {
            chunk_view* parent_; // exposition only

            constexpr explicit outer-iterator(chunk_view& parent); // exposition only
        public:
            using iterator_concept = input_iterator_tag;
            using difference_type = range_difference_t<V>;

            // [range.chunk.outer.value], class chunk_view::outer-iterator::value_type
            struct value_type;

            outer-iterator(outer-iterator&&) = default;
            outer-iterator& operator=(outer-iterator&&) = default;

            constexpr value_type operator*() const;
            constexpr outer-iterator& operator++();
            constexpr void operator++(int);

            friend constexpr bool operator==(const outer-iterator& x, default_sentinel_t);
```

```

    friend constexpr difference_type operator-(default_sentinel_t y, const outer-
        iterator& x)
        requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
    friend constexpr difference_type operator-(const outer-iterator& x,
        default_sentinel_t y)
        requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
};
}

```

```
constexpr explicit outer-iterator(chunk_view& parent);
```

1 *Effects:* Initializes `parent_` with `addressof(parent)`.

```
constexpr value_type operator*() const;
```

2 *Preconditions:* `*this == default_sentinel` is `false`.

3 *Returns:* `value_type(*parent_)`.

```
constexpr outer-iterator& operator++();
```

4 *Preconditions:* `*this == default_sentinel` is `false`.

5 *Effects:* Equivalent to:

```

    ranges::advance(*parent_->current_, parent_->remainder_,
        ranges::end(parent_->base_));
    parent_->remainder_ = parent_->n_;
    return *this;

```

```
constexpr void operator++(int);
```

6 *Effects:* Equivalent to `++*this`.

```
friend constexpr bool operator==(const outer-iterator& x, default_sentinel_t);
```

7 *Effects:* Equivalent to: `return *x.parent_->current_ == ranges::end(x.parent_ ->base_) && x.parent_->remainder_ != 0;`

```

friend constexpr difference_type operator-(default_sentinel_t y, const outer-iterator&
    x)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

```

8 *Effects:* Equivalent to:

```

    const auto dist = ranges::end(x.parent_->base_) - *x.parent_ -
        >current_;

    if (dist < x.parent_->remainder_) {
        return dist == 0 ? 0 : 1;
    }

    return div-ceil(dist - x.parent_->remainder_, x.parent_->n_) + 1;

```



```
friend constexpr difference_type operator-(const outer-iterator& x, default_sentinel_t
    y)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

9 *Effects:* Equivalent to: `return -(y - x);`

§ 24.7.?.4 Class `chunk_view::outer-iterator::value_type` [range.chunk.outer.value]

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        struct chunk_view<V>::outer-iterator::value_type : view_interface<value_type> {
            chunk_view* parent_; // exposition only

            constexpr explicit value_type(chunk_view& parent); // exposition only
        public:
            constexpr inner-iterator begin() const noexcept;
            constexpr default_sentinel_t end() const noexcept;

            constexpr auto size() const
                requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
        };
}
```

```
constexpr explicit value_type(chunk_view& parent);
```

1 *Effects:* Initializes `parent_` with `addressof(parent)`.

```
constexpr inner-iterator begin() const noexcept;
```

2 *Returns:* `inner-iterator(*parent_)`.

```
constexpr default_sentinel_t end() const noexcept;
```

3 *Returns:* `default_sentinel`.

```
constexpr auto size() const
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

4 *Effects:* Equivalent to: `return ranges::min(parent_>remainder_,
 ranges::end(parent_>base_) - *parent_>current_);`

§ 24.7.?.5 Class `chunk_view::inner-iterator` [range.chunk.inner.iter]

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        class chunk_view<V>::inner-iterator {
            chunk_view* parent_; // exposition only

            constexpr explicit inner-iterator(chunk_view& parent) noexcept; // exposition
            only
        public:
            using iterator_concept = input_iterator_tag;
```

```

using difference_type = range_difference_t<V>;
using value_type = range_value_t<V>;

inner-iterator(inner-iterator&&) = default;
inner-iterator& operator=(inner-iterator&&) = default;

constexpr const iterator_t<V>& base() const &;

constexpr range_reference_t<V> operator*() const;
constexpr inner-iterator& operator++();
constexpr void operator++(int);

friend constexpr bool operator==(const inner-iterator& x, default_sentinel_t);

friend constexpr difference_type operator-(default_sentinel_t y, const inner-
    iterator& x)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
friend constexpr difference_type operator-(const inner-iterator& x,
    default_sentinel_t y)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
};
}

```

```
constexpr explicit inner-iterator(chunk_view& parent) noexcept;
```

- 1 *Effects:* Initializes `parent_` with `addressof(parent)`.

```
constexpr const iterator_t<V>& base() const &;
```

- 2 *Effects:* Equivalent to: `return *parent_->current_;`

```
constexpr range_reference_t<V> operator*() const;
```

- 3 *Preconditions:* `*this == default_sentinel` is `false`.

- 4 *Effects:* Equivalent to: `return **parent_->current_;`

```
constexpr iterator& operator++();
```

- 5 *Preconditions:* `*this == default_sentinel` is `false`.

- 6 *Effects:* Equivalent to:

```

++*parent_->current_;
if (*parent_->current_ == ranges::end(parent_->base_)) {
    parent_->remainder_ = 0;
}
else {
    --parent_->remainder_;
}
return *this;

```

```
constexpr void operator++(int);
```

- 7 *Effects:* Equivalent to `++*this`.

```
friend constexpr bool operator==(const inner-iterator& x, default_sentinel_t);
```

8 *Returns:* `x.parent_ -> remainder_ == 0`.

```
friend constexpr difference_type operator-(default_sentinel_t y, const inner-iterator&
x)
requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

9 *Effects:* Equivalent to: `return ranges::min(x.parent_ -> remainder_,
ranges::end(x.parent_ -> base_) - *x.parent_ -> current_);`

```
friend constexpr difference_type operator-(const inner-iterator& x, default_sentinel_t
y)
requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

10 *Effects:* Equivalent to: `return -(y - x);`

§ 24.7.?.6 chunk_view for forward ranges [range.chunk.view.fwd]

```
namespace std::ranges {

template<view V>
requires forward_range<V>
class chunk_view<V> : public view_interface<chunk_view<V>> {
    V base_ = V(); // exposition only
    range_difference_t<V> n_ = 0; // exposition only
    template<bool> class iterator; // exposition only

public:
    chunk_view() requires default_initializable<V> = default;
    constexpr explicit chunk_view(V base, range_difference_t<V> n);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin() requires (!simple-view<V>) {
        return iterator<false>(this, ranges::begin(base_));
    }

    constexpr auto begin() const requires forward_range<const V> {
        return iterator<true>(this, ranges::begin(base_));
    }

    constexpr auto end() requires (!simple-view<V>) {
        if constexpr (common_range<V> && sized_range<V>) {
            auto missing = (n_ - ranges::distance(base_) % n_) % n_;
            return iterator<false>(this, ranges::end(base_), missing);
        }
        else if constexpr (common_range<V> && !bidirectional_range<V>) {
            return iterator<false>(this, ranges::end(base_));
        }
        else {
            return default_sentinel;
        }
    }
}
```

```
constexpr auto end() const requires forward_range<const V> {
    if constexpr (common_range<const V> && sized_range<const V>) {
        auto missing = (n_ - ranges::distance(base_) % n_) % n_;
        return iterator<true>(this, ranges::end(base_), missing);
    }
    else if constexpr (common_range<const V> && !bidirectional_range<const V>) {
        return iterator<true>(this, ranges::end(base_));
    }
    else {
        return default_sentinel;
    }
}

constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
};
```

```
constexpr explicit chunk_view(V base, range_difference_t<V> n);
```

- 1 *Preconditions:* `n > 0` is true.
- 2 *Effects:* Initializes `base_` with `std::move(base)` and `n_` with `n`.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

- 3 *Effects:* Equivalent to:

```
return to-unsigned-like(div-ceil(ranges::distance(base_), n_));
```

§ 24.7.7.7 Class template `chunk_view<V>::iterator` for forward ranges [range.chunk.fwd.iter]

```
namespace std::ranges {
    template<view V>
        requires forward_range<V>
    template<bool Const>
    class chunk_view<V>::iterator {
        using Parent = maybe_const<Const, chunk_view>;           // exposition only
        using Base = maybe_const<Const, V>;                     // exposition only

        iterator_t<Base> current_ = iterator_t<Base>();          // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>();              // exposition only
        range_difference_t<Base> n_ = 0;                          // exposition only
        range_difference_t<Base> missing_ = 0;                   // exposition only

        constexpr iterator(Parent* parent, iterator_t<Base> current, // exposition only
                           range_difference_t<Base> missing = 0);

    public:
        using iterator_category = input_iterator_tag;
        using iterator_concept = see_below;
        using value_type = decltype(views::take(subrange(current_, end_), n_));
        using difference_type = range_difference_t<Base>;
    };
}
```

```

iterator() = default;
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>
        && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

constexpr iterator_t<Base> base() const;

constexpr value_type operator*() const;
constexpr iterator& operator++();
constexpr iterator operator++(int);

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base>;

constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;

constexpr value_type operator[](difference_type n) const
    requires random_access_range<Base>;

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator==(const iterator& x, default_sentinel_t);

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

friend constexpr difference_type operator-(default_sentinel_t y, const iterator&
    x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t
    y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
};
}

```

1 *iterator::iterator_concept* is defined as follows:

- .1) — If *Base* models *random_access_range*, then *iterator_concept* denotes *random_access_iterator_tag*.
- .2) — Otherwise, if *Base* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- .3) — Otherwise, *iterator_concept* denotes *forward_iterator_tag*.

```
constexpr iterator(Parent* parent, iterator_t<Base> current,  
                  range_difference_t<Base> missing = 0);
```

- 2 *Effects*: Initializes *current_* with *current*, *end_* with *ranges::end(parent->base_)*, *n_* with *parent->n_*, and *missing_* with *missing*.

```
constexpr iterator(iterator<!Const> i)  
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>  
              && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

- 3 *Effects*: Initializes *current_* with *std::move(i.current_)*, *end_* with *std::move(i.end_)*, *n_* with *i.n_*, and *missing_* with *i.missing_*.

```
constexpr iterator_t<Base> base() const;
```

- 4 *Returns*: *current_*.

```
constexpr value_type operator*() const;
```

- 5 *Preconditions*: *current_ != end_* is *true*.

- 6 *Returns*: *views::take(subrange(current_, end_), n_)*

```
constexpr iterator& operator++();
```

- 7 *Preconditions*: *current_ != end_* is *true*.

- 8 *Effects*: Equivalent to:

```
    missing_ = ranges::advance(current_, n_, end_);  
    return *this;
```

```
constexpr iterator operator++(int);
```

- 9 *Effects*: Equivalent to:

```
    auto tmp = *this;  
    ++*this;  
    return tmp;
```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

- 10 *Effects*: Equivalent to:

```

        ranges::advance(current_, missing_ - n_);
        missing_ = 0;
        return *this;

```

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

11 *Effects:* Equivalent to:

```

    auto tmp = *this;
    --*this;
    return tmp;

```

```
constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
```

12 *Preconditions:* If x is positive, `ranges::distance(current_, end_) > n_ * (x - 1)` is `true`. [*Note 1:* If x is negative, the *Effects:* paragraph implies a precondition. — *end note*]

13 *Effects:* Equivalent to:

```

    if (x > 0) {
        missing_ = ranges::advance(current_, n_ * x, end_);
    }
    else if (x < 0) {
        ranges::advance(current_, n_ * x + missing_);
        missing_ = 0;
    }
    return *this;

```

```
constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;
```

14 *Effects:* Equivalent to: `return *this += -x;`

```
constexpr value_type operator[](difference_type n) const
    requires random_access_range<Base>;
```

15 *Returns:* `*(this + n)`.

```
friend constexpr bool operator==(const iterator& x, const iterator& y);
```

16 *Returns:* `x.current_ == y.current_`.

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

17 *Returns:* `x.current_ == x.end_;`

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

18 *Returns:* `x.current_ < y.current_`.

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

19 *Effects:* Equivalent to: `return y < x;`

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

20 *Effects:* Equivalent to: `return !(y < x);`

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

21 *Effects:* Equivalent to: `return !(x < y);`

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
           three_way_comparable<iterator_t<Base>>;
```

22 *Returns:* `x.current_ <=> y.current_.`

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
```

23 *Effects:* Equivalent to:

```
    auto r = i;
    r += n;
    return r;
```

```
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
```

24 *Effects:* Equivalent to:

```
    auto r = i;
    r -= n;
    return r;
```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
```

25 *Returns:* `(x.current_ - y.current_ + x.missing_ - y.missing_) / x.n_.`

```
friend constexpr difference_type operator-(default_sentinel_t y, const iterator& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
```

26 *Returns:* `div-ceil(x.end_ - x.current_, x.n_).`

```
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
```

27 *Effects:* Equivalent to: `return -(y - x);`

§ 5.3 slide

Add the following subclause to 24.7 [\[range.adaptors\]](#).

§ 24.7.? Slide view [\[range.slide\]](#)

§ 24.7.?.1 Overview [\[range.slide.overview\]](#)

- 1 `slide_view` takes a view and a number N and produces a view whose M^{th} element is a view over the M^{th} through $(M + N - 1)^{\text{th}}$ elements of the original view. If the original view has fewer than N elements, the resulting view is empty.
- 2 The name `views::slide` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions E and N , the expression `views::slide( $E$ ,  $N$ )` is expression-equivalent to `slide_view( $E$ ,  $N$ )`.

[Example 1:

```
vector v = {1, 2, 3, 4};

for (auto i : v | views::slide(2)) {
    cout << '[' << i[0] << ', ' << i[1] << "]" "; // prints: [1, 2] [2, 3] [3, 4]
}
```

— end example]

§ 24.7.?.2 Class template `slide_view` [\[range.slide.view\]](#)

```
namespace std::ranges {

template<class V>
    concept slide-caches-nothing = random_access_range<V> && sized_range<V>; //
        exposition only

template<class V>
    concept slide-caches-last = !slide-caches-nothing<V> && bidirectional_range<V> &&
        common_range<V>; // exposition only

template<class V>
    concept slide-caches-first = !slide-caches-nothing<V> && !slide-caches-last<V>; //
        exposition only

template<forward_range V>
    requires view<V>
class slide_view : public view_interface<slide_view<V>>{
    V base_ = V(); // exposition only
    range_difference_t<V> n_ = 0; // exposition only

    template<bool> class iterator; // exposition only
    class sentinel; // exposition only

public:
    slide_view() requires default_initializable<V> = default;
    constexpr explicit slide_view(V base, range_difference_t<V> n);
```

```
constexpr auto begin()
    requires (!simple-view<V> && slide-caches-nothing<const V>));
constexpr auto begin() const requires slide-caches-nothing<const V>;

constexpr auto end()
    requires (!simple-view<V> && slide-caches-nothing<const V>));
constexpr auto end() const requires slide-caches-nothing<const V>;

constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
};

template<class R>
    slide_view(R&& r, range_difference_t<R>) -> slide_view<views::all_t<R>>;
}
```

```
constexpr explicit slide_view(V base, range_difference_t<V> n);
```

- 1 *Effects:* Initializes *base_* with `std::move(base)` and *n_* with *n*.

```
constexpr auto begin()
    requires (!simple-view<V> && slide-caches-nothing<const V>));
```

- 2 *Returns:*

- (2.1) — If *v* models *slide-caches-first*, `iterator<false>(ranges::begin(base_), ranges::next(ranges::begin(base_), n_ - 1, ranges::end(base_)), n_)`.
- (2.2) — Otherwise, `iterator<false>(ranges::begin(base_), n_)`.

- 3 *Remarks:* In order to provide the amortized constant-time complexity required by the range concept, this function caches the result within the `slide_view` for use on subsequent calls when *v* models *slide-caches-first*.

```
constexpr auto begin() const requires slide-caches-nothing<const V>;
```

- 4 *Returns:* `iterator<true>(ranges::begin(base_), n_)`.

```
constexpr auto end()
    requires (!simple-view<V> && slide-caches-nothing<const V>));
```

- 5 *Returns:*

- (5.1) — If *v* models *slide-caches-nothing*, `iterator<false>(ranges::begin(base_) + range_difference_t<V>(size()), n_)`;
- (5.2) — Otherwise, if *v* models *slide-caches-last*, `iterator<false>(ranges::prev(ranges::end(base_), n_ - 1, ranges::begin(base_)), n_)`;
- (5.3) — Otherwise, if *v* models *common_range*, `iterator<false>(ranges::end(base_), ranges::end(base_), n_)`;
- (5.4) — Otherwise, `sentinel(ranges::end(base_))`.

- 6 *Remarks:* In order to provide the amortized constant-time complexity required by the range concept, this function caches the result within the `slide_view` for use on subsequent calls when `V` models *slide-caches-last*.

```
constexpr auto end() const requires slide-caches-nothing<const V>;
```

- 7 *Returns:* `begin() + range_difference_t<const V>(size())`.

```
constexpr auto size() requires sized_range<V>;  
constexpr auto size() const requires sized_range<const V>;
```

- 8 *Effects:* Equivalent to:

```
auto sz = ranges::distance(base_) - n_ + 1;  
if (sz < 0) sz = 0;  
return to-unsigned-like(sz);
```

§ 24.7.3.3 Class template `slide_view::iterator` [range.slide.iterator]

```
namespace std::ranges {  
    template<forward_range V>  
        requires view<V>  
        template<bool Const>  
        class slide_view<V>::iterator {  
            using Base = maybe-const<Const, V>; // exposition only  
            iterator_t<Base> current_ = iterator_t<Base>(); // exposition only  
            iterator_t<Base> last_ele_ = iterator_t<Base>(); // exposition only,  
                present only if Base models slide-caches-first  
            range_difference_t<Base> n_ = 0; // exposition only  
  
            constexpr iterator(iterator_t<Base> current, range_difference_t<Base> n) //  
                exposition only  
                requires (!slide-caches-first<Base>);  
  
            constexpr iterator(iterator_t<Base> current, iterator_t<Base> last_ele,  
                range_difference_t<Base> n) // exposition only  
                requires slide-caches-first<Base>;  
        public:  
            using iterator_category = input_iterator_tag;  
            using iterator_concept = see below;  
            using value_type = decltype(views::counted(current_, n_));  
            using difference_type = range_difference_t<Base>;  
  
            iterator() = default;  
            constexpr iterator(iterator<!Const> i)  
                requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;  
  
            constexpr auto operator*() const;  
            constexpr iterator& operator++();  
            constexpr iterator operator++(int);  
  
            constexpr iterator& operator--() requires bidirectional_range<Base>;  
            constexpr iterator operator--(int) requires bidirectional_range<Base>;  
  
            constexpr iterator& operator+=(difference_type x)
```

```

    requires random_access_range<Base>;
constexpr iterator& operator--(difference_type x)
    requires random_access_range<Base>;

constexpr auto operator[](difference_type n) const
    requires random_access_range<Base>;

friend constexpr bool operator==(const iterator& x, const iterator& y);

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
};
}

```

1 *iterator*::iterator_concept is defined as follows:

- .1) — If *Base* models `random_access_range`, then `iterator_concept` denotes `random_access_iterator_tag`.
- .2) — Otherwise, if *Base* models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- .3) — Otherwise, `iterator_concept` denotes `forward_iterator_tag`.

2 If the invocation of any non-const member function of *iterator* exits via an exception, the iterator acquires a singular value.

```

constexpr iterator(iterator_t<Base> current, range_difference_t<Base> n)
    requires (!slide_caches_first<Base>);

```

3 *Effects*: Initializes *current_* with *current* and *n_* with *n*.

```

constexpr iterator(iterator_t<Base> current, iterator_t<Base> last_ele,
    range_difference_t<Base> n)
    requires slide_caches_first<Base>;

```

4 *Effects*: Initializes *current_* with *current*, *last_ele_* with *last_ele*, and *n_* with *n*.

```
constexpr iterator(iterator<!Const> i)
    requires Const && (convertible_to<iterator_t<V>, iterator_t<Base>>);
```

- 5 *Effects:* Initializes *current_* with `std::move(i.current_)` and *n_* with `i.n_`. [*Note 1:* `iterator<true>` can only be formed when *Base* models *slide-caches-nothing*, in which case *last_ele_* is not present. — end note]

```
constexpr auto operator*() const;
```

- 6 *Returns:* `views::counted(current_, n_)`.

```
constexpr iterator& operator++();
```

- 7 *Preconditions:* *current_* and *last_ele_* (if present) are incrementable.
- 8 *Postconditions:* *current_* and *last_ele_* (if present) are each equal to `ranges::next(i)`, where *i* is the value of that data member before the call.
- 9 *Returns:* `*this`.

```
constexpr iterator operator++(int);
```

- 10 *Effects:* Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

- 11 *Preconditions:* *current_* and *last_ele_* (if present) are decrementable.
- 12 *Postconditions:* *current_* and *last_ele_* (if present) are each equal to `ranges::prev(i)`, where *i* is the value of that data member before the call.
- 13 *Returns:* `*this`.

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

- 14 *Effects:* Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
```

- 15 *Preconditions:* *current_* + *x* and *last_ele_* + *x* (if present) have well-defined behavior.
- 16 *Postconditions:* *current_* and *last_ele_* (if present) are each equal to *i* + *x*, where *i* is the value of that data member before the call.

17 *Returns: `*this`.*

```
constexpr iterator& operator--(difference_type x)
    requires random_access_range<Base>;
```

18 *Preconditions: `current_ - x` and `last_ele_ - x` (if present) have well-defined behavior.*

19 *Postconditions: `current_` and `last_ele_` (if present) are each equal to `i - x`, where `i` is the value of that data member before the call.*

20 *Returns: `*this`.*

```
constexpr auto operator[](difference_type n) const
    requires random_access_range<Base>;
```

21 *Effects: Equivalent to: `return views::counted(current_ + n, n_);`*

```
friend constexpr bool operator==(const iterator& x, const iterator& y);
```

22 *Returns: If `last_ele_` is present, `x.last_ele_ == y.last_ele_`; otherwise, `x.current_ == y.current_`.*

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

23 *Returns: `x.current_ < y.current_`.*

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

24 *Effects: Equivalent to: `return y < x;`*

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

25 *Effects: Equivalent to: `return !(y < x);`*

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

26 *Effects: Equivalent to: `return !(x < y);`*

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;
```

27 *Returns: `x.current_ <=> y.current_`.*

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
```

28 *Effects: Equivalent to:*

```

auto r = i;
r += n;
return r;

```

```

friend constexpr iterator operator-(const iterator& i, difference_type n)
requires random_access_range<Base>;

```

29 *Effects:* Equivalent to:

```

auto r = i;
r -= n;
return r;

```

```

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

```

30 *Returns:* If `last_ele_` is present, `x.last_ele_ - y.last_ele_`; otherwise, `x.current_ - y.current_`.

§ 24.7.4 Class `slide_view::sentinel` [range.slide.sentinel]

```

namespace std::ranges {
    template<forward_range V>
    requires view<V>
    class slide_view<V>::sentinel {
        sentinel_t<V> end_ = sentinel_t<V>();           // exposition only
        constexpr explicit sentinel(sentinel_t<V> end); // exposition only
    public:
        sentinel() = default;

        friend constexpr bool operator==(const iterator<false>& x, const sentinel& y);

        friend constexpr range_difference_t<V>
        operator-(const iterator<false>& x, const sentinel& y)
        requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

        friend constexpr range_difference_t<V>
        operator-(const sentinel& y, const iterator<false>& x)
        requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
    };
}

```

1 [Note 1: `sentinel` is only used when `slide-caches-first<V>` is `true`. — end note]

```

constexpr explicit sentinel(sentinel_t<V> end);

```

2 *Effects:* Initializes `end_` with `end`.

```

friend constexpr bool operator==(const iterator<false>& x, const sentinel& y);

```

3 *Returns:* `x.last_ele_ == y.end_`.

```

friend constexpr range_difference_t<V>
operator-(const iterator<false>& x, const sentinel& y)
requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

```

4 *Returns: `x.last_ele_ - y.end_`.*

```
friend constexpr range_difference_t<V>
operator-(const sentinel& y, const iterator<false>& x)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

5 *Returns: `y.end_ - x.last_ele_`.*

§ 5.4 Feature-test macro

Add the following macro definitions to 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_chunk 20XXXXL // also in <ranges>
#define __cpp_lib_ranges_slide 20XXXXL // also in <ranges>
```

§ 6 References

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.
<https://wg21.link/p2214r0>

[P2321R2] Tim Song. 2021-06-11. zip.
<https://wg21.link/p2321r2>